

② Tuple :-

- properties ⇒ 'Tuple' is one of the pre-defined class and treated as list data type.
 - ⇒ The purpose of tuple data type is that "to store multiple values either of same type (OR) Different type or Both the types in single object with unique and duplicate values".
 - ⇒ The values/Elements of tuple must be stored/organized with Braces () and values must be separated by comma.
 - ⇒ An object of tuple maintains insertion order.
 - ⇒ An object of tuple, we can perform Both Indexing and Slicing operations.

=> An object of tuple belongs to immutable.

=> w.r.t tuple class, we can create 2 types of tuple objects.

They are:-

(a) Empty tuples

(b) non-empty tuples.

(a) Empty tuple:- An empty tuple is one, which does not contain any elements whose length is 0.

Syntax: Varname = ()
(OR)
Varname = tuple()

(b) Non-empty tuple:- An non-empty tuple is one, which contain elements and whose length is > 0

Syntax: Varname = (val1, val2, ..., val-n)
(OR)

Syntax: Varname = val1, val2, ..., val-n
(OR)

Varname = tuple(object)

Note:- The functionality of tuple is exactly similar to functionality of list but list object belongs to mutable and tuple object belongs to immutable

Examples:-

```
>>> t1 = (10, 20, 30, 10, 50, 60, 70)
```

```
>>> print(t1, type(t1)) --- (10, 20, 30, 10, 50, 60, 70)
                                     <class 'tuple'>
```

```
>>> t2 = (100, "Pravish", 33.333, "Hyd")
```

```
>>> print(t2, type(t2)) --- (100, 'Pravish', 33.33, 'Hyd')
                                     <class 'tuple'>
```

```
>>> len(t1) --- 7
```

```
>>> len(t2) --- 4.
```

```

>>> t3 = 100, 'Rossum', 44.44, 'PYTHON'
>>> print(t3, type(t3)) — — — (100, 'Rossum', 44.44, 'PYTHON')
                                     <class 'tuple'>
=
>>> t1 = ()
>>> print(t1, type(t1)) — — — () <class 'tuple'>
>>> len(t1) — — — 0
>>> t2 = tuple()
>>> print(t2, type(t2)) — — — () <class 'tuple'>
>>> len(t2) — — — 0.
=
>>> t = (100, "Rossum", 44.44, "PYTHON")
>>> print(t, type(t), id(t)) — — — (100, 'Rossum', 44.44,
                                     'PYTHON')
                                     <class 'tuple'> 2605398318096

>>> t[0] — — — 100
>>> t[1] — — — 'Rossum'
>>> t[-1] — — — 'PYTHON'
>>> t[0] = 200 — — — Type error: ''

=
>>> l1 = [100, "Rossum", 44.44, "PYTHON"]
>>> print(l1, type(l1)) — — — [100, 'Rossum', 44.44, 'PYTHON']
                                     <class 'list'>
>>> t1 = tuple(l1)
>>> print(t1, type(t1)) — — — (100, 'Rossum', 44.44, 'PYTHON')
                                     <class 'tuple'>
>>> l2 = list(t1)
>>> print(l2, type(l2)) — — — [100, 'Rossum', 44.44,
                                     'PYTHON'] <class 'list'>
=
>>> t = (100, "Rossum", 44.44, "PYTHON")
>>> print(t, type(t)) — — — (100, 'Rossum', 44.44, 'PYTHON')
                                     <class 'tuple'>
>>> t[0:3] — — — (100, 'Rossum', 44.44)
>>> t[0:-1] — — — ('PYTHON', 44.44, 'Rossum', 100)

```

Special points

```

>>> a = 10
>>> t = tuple(a)
>>> t = tuple(a, )
>>> t = tuple((a,))
>>> t = tuple([a])
>>> print(t, type(t)) — — — (10,) <class 'tuple'>
      (OR)

```

Type error: 'int' object is not iterable

```

>>> b = 100
>>> t = (b, )
>>> print(t, type(t)) — — — (100,) <class 'tuple'>

```

```

>>> b = 12.34
>>> t = (b)
>>> print(t, type(t)) — — — 12.34 <class 'float'>
>>> t = (b, )
>>> print(t, type(t)) — — — (12.34,) <class 'tuple'>

```

* Pre-defined function in tuple :-

- we know that on the object of tuple we can perform both indexing and slicing operations.
- Along with these operations, we can also perform other operations by using the following pre-defined functions.

- ① index()
- ② Count()

Example

```

>>> t1 = (10, "Rs", 45.67)
>>> print(t1, type(t1)) — — — (10, 'Rs', 45.67) <class 'tuple'>
>>> t1.index(10) — — — 0
>>> t1.index("Rs") — — — 1
>>> t1 = (10, "Rs", 45.67)

```

```

>>> print(t1, type(t1)) -- (10, 'rs', 45.67) <class 'tuple'>
>>> t1.count(10) -- 1
>>> t1.count(100) -- 0
>>> t1 = (10, 0, 10, 10, 20, 0, 10)
>>> print(t1, type(t1)) -- (10, 0, 10, 10, 20, 0, 10)
                                <class 'tuple'>
>>> t1.count(10) -- 4
>>> t1.count(0) -- 2
>>> t1.count(100) -- 0.
=

```

* The function not present in tuple.

- append()
- insert()
- remove()
- clear()
- pop(index)
- pop()
- reverse()
- sort()
- copy()
- extend()

Note: By using del operator we can't delete value of tuple object by using indexing and slicing but we can delete entire object.

Examples

```

>>> t1 = (10, -34, 0, 10, 23, 56, 76, 21)
>>> print(t1, type(t1)) -- (10, -34, 0, 10, 23, 56, 76, 21)
                                <class 'tuple'>
[ >>> del t1[0] -- Type error.
  >>> del t1[0:4] -- Type error
]

```

```
>>> del t1 # Here we are removing complete object.
>>> print(t1, type(t1)) - - - Name error
```

Most Important

- Sorted() :- This function is used for sorting the data of immutable object tuple and gives the sorted data in the form of list.

Syntax : `listobj = sorted(tuple object)`

• Examples

```
>>> t1 = (10, 23, -56, -1, 13, 15, 6, -2)
>>> print(t1, type(t1)) - - - (10, 23, -56, -1, 13, 15, 6, -2)
                                     <class 'tuple'>
>>> t1.sort() - - - Attribute error.
>>> x = sorted(t1)
>>> print(x, type(x)) - - - [-56, -2, -1, 6, 10, 13, 15, 23]
                                     <class 'list'>
>>> print(t1, type(t1)) - - - (10, 23, -56, -1, 13, 15, 6, -2)
                                     <class 'tuple'>
>>> t1 = tuple(x) # converted sorted list into tuples.
>>> print(t1, type(t1)) - - - (-56, -2, -1, 6, 10, 13, 15, 23)
                                     <class 'tuple'>
>>> t2 = t1[::-1]
>>> print(t2, type(t2)) - - - (23, 15, 13, 10, 6, -1, -2, -56)
                                     (OR)
                                     <class 'tuple'>
>>> t1 = (10, -4, 12, 34, 16, -6, 0, 15)
>>> print(t1, type(t1)) - - - (10, -4, 12, 34, 16, -6, 0, 15)
                                     <class 'tuple'>
>>> l1 = list(t1)
>>> print(l1, type(l1)) - - - [10, -4, 12, 34, 16, -6, 0, 15]
                                     <class 'list'>
```

tuple: the purpose of tuple data type is that "to store multiple values either of same type"

```
In [1]: t1=(10,20,30,10,50,60,70)
        print(t1,type(t1))
```

```
(10, 20, 30, 10, 50, 60, 70) <class 'tuple'>
```

```
In [2]: t2=(100,"travis",33.33,"hyd")
        print(t2,type(t2))
```

```
(100, 'travis', 33.33, 'hyd') <class 'tuple'>
```

```
In [3]: print(len(t1))
        print(len(t2))
```

7
4

```
In [4]: t3=(100,"rosum",44.44,"python")
        print(t3,type(t3))
```

(100, 'rosum', 44.44, 'python') <class 'tuple'>

```
In [5]: t1=()
        print(t1,type(t1))
        len(t1)
```

() <class 'tuple'>

Out[5]: 0

```
In [7]: t2=tuple()
        print(t2,type(t2))
        len(t2)
```

() <class 'tuple'>

Out[7]: 0

```
In [8]: t=(100,"rosum",44.44,"python")
        print(t,type(t),id(t))
```

(100, 'rosum', 44.44, 'python') <class 'tuple'> 2746342749616

```
In [9]: t[0]
```

Out[9]: 100

```
In [10]: t[1]
```

Out[10]: 'rosum'

```
In [11]: t[-1]
```

Out[11]: 'python'

```
In [12]: t[0]=200
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[12], line 1
----> 1 t[0]=200

TypeError: 'tuple' object does not support item assignment
```

```
In [17]: l1=[100,"rosum",44.44,"python"]
        print(l1,type(l1))
```

[100, 'rosum', 44.44, 'python'] <class 'list'>

```
In [19]: t1=tuple(l1)
        print(t1,type(t1))
```

(100, 'rosum', 44.44, 'python') <class 'tuple'>

```
In [20]: l2=list(t1)
         print(l2,type(l2))
```

[100, 'rosum', 44.44, 'python'] <class 'list'>

```
In [21]: t=(100,"rosum",44.44,"python")
         print(t,type(t))
         print(t[0:3])
         print(t[::-1])
```

(100, 'rosum', 44.44, 'python') <class 'tuple'>

(100, 'rosum', 44.44)

('python', 44.44, 'rosum', 100)

```
In [22]: #special points:-
         a=10
         t=tuple(a)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[22], line 3
      1 #special points:-
      2 a=10
----> 3 t=tuple(a)

TypeError: 'int' object is not iterable
```

```
In [23]: t=tuple(a,)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[23], line 1
----> 1 t=tuple(a,)

TypeError: 'int' object is not iterable
```

```
In [25]: t=tuple([a])
         print(t,type(t))
```

(10,) <class 'tuple'>

```
In [26]: b=100
         t=(b,)
         print(t,type(t))
```

(100,) <class 'tuple'>

```
In [27]: b=12.34
         t=(b)
         print(t,type(t))
```

12.34 <class 'float'>

```
In [28]: t=(b,)
         print(t,type(t))
```

(12.34,) <class 'tuple'>

```
In [30]: #predefined function in tuple:=
         #1.index() 2.count()
         t1=(10,"RS",45.67)
         print(t1,type(t1))
```

```
(10, 'RS', 45.67) <class 'tuple'>
```

```
In [31]: t1.index(10)
```

```
Out[31]: 0
```

```
In [32]: t1.index("RS")
```

```
Out[32]: 1
```

```
In [33]: t1=(10,"RS",45.67)
print(t1,type(t1))
```

```
(10, 'RS', 45.67) <class 'tuple'>
```

```
In [34]: t1.count(10)
```

```
Out[34]: 1
```

```
In [35]: t1.count(100)
```

```
Out[35]: 0
```

```
In [36]: t1=(10,0,10,10,20,0,10)
print(t1,type(t1))
```

```
(10, 0, 10, 10, 20, 0, 10) <class 'tuple'>
```

```
In [37]: t1.count(10)
```

```
Out[37]: 4
```

```
In [38]: t1.count(0)
```

```
Out[38]: 2
```

```
In [39]: t1.count(100)
```

```
Out[39]: 0
```

```
In [40]: #del operator
t1=(10,-34,0,10,23,56,76,21)
print(t1,type(t1))
```

```
(10, -34, 0, 10, 23, 56, 76, 21) <class 'tuple'>
```

```
In [41]: del t1[0]
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[41], line 1
----> 1 del t1[0]

TypeError: 'tuple' object doesn't support item deletion
```

```
In [42]: del t1[0:4]
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[42], line 1
----> 1 del t1[0:4]

TypeError: 'tuple' object does not support item deletion
```

```
In [44]: del t1
        print(t1,type(t1))
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[44], line 1
----> 1 del t1
      2 print(t1,type(t1))

NameError: name 't1' is not defined
```

```
In [45]: #sorted()
        #syntax:-      listobj=sorted(tupleobject)
        t1=(10,23,-56,-1,13,15,6,-2)
        print(t1,type(t1))

(10, 23, -56, -1, 13, 15, 6, -2) <class 'tuple'>
```

```
In [46]: t1.sort()
```

```
-----
AttributeError                            Traceback (most recent call last)
Cell In[46], line 1
----> 1 t1.sort()

AttributeError: 'tuple' object has no attribute 'sort'
```

```
In [47]: x=sorted(t1)
        print(x,type(x))

[-56, -2, -1, 6, 10, 13, 15, 23] <class 'list'>
```

```
In [48]: print(t1,type(t1))

(10, 23, -56, -1, 13, 15, 6, -2) <class 'tuple'>
```

```
In [49]: t1=tuple(x)
        print(t1,type(t1))

(-56, -2, -1, 6, 10, 13, 15, 23) <class 'tuple'>
```

```
In [50]: t2=t1[::-1]
        print(t2,type(t2))

(23, 15, 13, 10, 6, -1, -2, -56) <class 'tuple'>
```

```
In [51]: t1=(10,-4,12,34,16,-6,0,15)
        print(t1,type(t1))

(10, -4, 12, 34, 16, -6, 0, 15) <class 'tuple'>
```

```
In [52]: l1=list(t1)
        print(l1,type(l1))

[10, -4, 12, 34, 16, -6, 0, 15] <class 'list'>
```

```
In [53]: l1.sort()
         print(l1,type(l1))
```

```
[-6, -4, 0, 10, 12, 15, 16, 34] <class 'list'>
```

```
In [54]: t1=tuple(l1)
         print(t1,type(t1))
```

```
(-6, -4, 0, 10, 12, 15, 16, 34) <class 'tuple'>
```

```
In [55]: t1=t1[::-1]
         print(t1,type(t1))
```

```
(34, 16, 15, 12, 10, 0, -4, -6) <class 'tuple'>
```

Q: What is the difference between a list and a tuple in Python?

Answer: 1.Mutability:

List: Mutable – elements can be added, removed, or changed.

Tuple: Immutable – once defined, elements cannot be modified.

2.Syntax:

List: Defined using square brackets, e.g., my_list = [1, 2, 3]

Tuple: Defined using parentheses, e.g., my_tuple = (1, 2, 3)

3.Performance:

List: Slightly slower due to mutability.

Tuple: Faster in iteration and access due to immutability.

4.Use Cases:

List: Suitable when data needs to be modified.

Tuple: Suitable for fixed data or to ensure data integrity.

5.Memory Usage:

Tuple: Generally uses less memory than a list with the same elements.

6.Methods:

List: Has more built-in methods (e.g., append(), remove(), pop()).

Tuple: Fewer methods available (e.g., count(), index()).

7.Hashability:

List: Not hashable – cannot be used as keys in dictionaries.

Tuple: Hashable (if it contains only immutable elements) – can be used as dictionary keys.

In []: